Streamlit Web Application Codebase Complete Architecture & File Walkthrough

Deep Dive into All Python Files in project/streamlit_app
8th Semester Natural Language Processing (NLP)

Course & Project Details

Course:	Natural Language Processing (NLP) - 8th Semester CS
Application Directory:	project/streamlit_app/
Files Covered:	app.py, contracts.py, engine.py, transport.py, ingestion.py, etc.
Coverage:	Role of file, Architectural responsibility, Code structure & Line-by-Line Roman Urdu Breakdown
Document Scope:	Part 3: Production Codebase Technical Report & Viva Defense

Guide Ka Maqsad (Purpose)

Yeh document NLP 8th Semester project ki mukammal samajh, multi-agent LLM pipeline, codebase architecture, aur un tamam challenges ke practical solutions ko Roman Urdu aur English mein detail se explain karne ke liye tayyar kiya gaya hai. Is se project viva, evaluation aur presentation ke har sawal ka mukammal jawaz milta hai.

Codebase Architecture Overview

Streamlit application ka codebase modular clean-architecture design follow karta hai. Presentation layer (`app.py`), Data contracts (`contracts.py`), Pipeline engine (`engine.py`), LLM Transport (`transport.py`), Data ingestion (`ingestion.py`), Quality Audit (`review.py`), Publishing exports (`exports.py`), aur Security guardrails (`security.py`) bilkul decoupled hain. Neechay har file ka mukammal role, key code patterns, aur Roman Urdu mein tafseeli wazahat pesh ki gayi hai.

File: app.py

>> Module Role: Streamlit Frontend User Interface & Job Manager

Architectural Role & Work of app.py

app.py pure application ka web user interface (UI) control karta hai. Iska kaam user se input lena, AI Model Provider (Gemini, Groq, Cerebras) select karwana, asynchronous background thread mein generation start karna, real-time progress updates dikhana, aur aane wale result ko Markdown, JSON, aur PDF mein export karwana hai.

Key Functions & Logic:
- `setting(name)`: Secrets lookup karta hai pehle `os.getenv` se phir `st.secrets` se taake Streamlit Cloud par credentials seamlessly milen.
- `active_providers()`: Check karta hai ke konsi API keys (.env ya secrets mein) mojood hain aur unhi models ko dropdown mein show karta hai.
- `job_manager()`: Background threading mechanism (`threading.Thread`) use karta hai taake LLM calls ke doran Streamlit UI freeze na ho.
- `options = sidebar()`: Topic, audience, target words, language (English/Urdu/Roman Urdu), aur output choice (Outline ya Complete Article) ke widgets render karta hai.
- `show_results(result)`: Outline, FAQ, SEO metadata, Audit issues, aur Download buttons (PDF, Markdown, JSON) display karta hai.

Representative Code Listing (app.py):

```
import hmac
import json
import os
import time
from pathlib import Path
from queue import Empty
import streamlit as st
from dotenv import load_dotenv
from planner.exports import content_markdown, download_markdown, make_pdf, safe_markdown, STATUS_LAB
from planner.jobs import JobManager
from planner.security import validate_upload
from planner.sample import sample_result

load_dotenv(Path(__file__).with_name(".env"))
st.set_page_config(page_title="SEO Content Planner", layout="wide", initial_sidebar_state="auto")


def setting(name):
    value = os.getenv(name, "")
    if not value:
        try:
            if hasattr(st, "secrets") and name in st.secrets:
                value = st.secrets[name]
            elif hasattr(st, "secrets"):
                value = st.secrets.get(name, "")
        except Exception:
            pass
    if isinstance(value, str):
        return value.strip().strip('"').strip("'")
    return value
```

```
def styles():
    st.markdown("""<style>
    .block-container {max-width:1200px;padding-top:2.4rem;padding-bottom:4rem;}
    [data-testid="stSidebar"] {border-right:1px solid rgba(148,163,184,.13);}
    h1 {font-size:2.5rem!important;letter-spacing:-.06rem;font-weight:650!important;}
    h2 {letter-spacing:-.025rem;}
    .eyebrow {color:#A5B4FC;font-size:.75rem;letter-spacing:.16rem;font-weight:650;margin-bottom:.7r
    .lede {color:#9AA8BC;font-size:1.05rem;max-width:700px;line-height:1.7;margin-bottom:1.7rem;}
    .step {border-top:2px solid #334155;padding:1rem 0;color:#9AA8BC;font-size:.9rem;}
    .step strong {color:#E6EAF2;display:block;margin-bottom:.3rem;font-size:1rem;}
# ... [Truncated: 369 more lines in full code file]
```

File: planner/contracts.py

>> Module Role: Pydantic v2 Data Models & Validation Contracts

Architectural Role & Work of planner/contracts.py

contracts.py pure project ki data integrity ka zamin hai. Is file mein har stage ke inputs aur outputs ke strict Pydantic v2 data models define kiye gaye hain. LLMs structured output return karte hain jo inhi models ke mutabiq validate hoti hai.

Key Classes & Logic:
- `StrictModel`: Base model jismein `extra='forbid'` laga hai taake LLM apni taraf se koi extra field invent na kar sake.
- `Intake`: User request ka schema (topic, audience, instructions, target_words, output_language).
- `Article`: Complete article ka schema (title, meta_description, introduction, sections, faqs, output_language).
- `Section`: Article ke har section ka schema (heading, body, chunk_ids).
- `FAQ`: Question, answer, aur associated chunk_ids.
- `count_words(text)`: Multilingual word counting function jo spaces ke mutabiq words count karta hai.
- `article_text(article)`: Article ke tamam hisson (intro, sections, faqs) ko ek string mein jodta hai taake word count aur audit check ho sake.

Representative Code Listing (planner/contracts.py):

```
import re
from pydantic import BaseModel, ConfigDict, Field
from typing import Literal


class Intake(BaseModel):
    model_config = ConfigDict(extra="forbid")
    topic: str = Field(min_length=2, max_length=300)
    audience: str = Field(default="General audience", max_length=500)
    instructions: str = Field(default="", max_length=4000)


class Section(BaseModel):
    model_config = ConfigDict(extra="forbid")
    heading: str
    body: str = Field(min_length=20)
    chunk_ids: list[str] = Field(default_factory=list)


class FAQ(BaseModel):
    model_config = ConfigDict(extra="forbid")
    question: str
    answer: str
    chunk_ids: list[str] = Field(default_factory=list)


class Article(BaseModel):
    model_config = ConfigDict(extra="forbid")
    title: str
    meta_description: str
    introduction: str
    sections: list[Section] = Field(min_length=2, max_length=8)
    faqs: list[FAQ] = Field(default_factory=list, min_length=0, max_length=8)
```

```
    output_language: Literal["English", "Urdu", "Roman Urdu"]


def count_words(text):
    return len(re.findall(r"\S+", text))


def article_text(article):
    parts = [article["introduction"]]
# ... [Truncated: 39 more lines in full code file]
```

File: planner/engine.py

>> Module Role: Master Pipeline Coordinator & Resilient Embeddings

Architectural Role & Work of planner/engine.py

engine.py pipeline ka central brain hai jo tamam stages ko coordinate karta hai. Is file mein resilient zero-download embeddings, concurrency (`asyncio.gather`), Urdu script validation, aur exact word count trimming implement kiye gaye hain.

Key Components & Logic:
- `FallbackTokenizer` & `FallbackEmbeddingModel`: Streamlit Cloud par jab Hugging Face offline ho ya block ho, to yeh model deterministic MD5 n-gram hashing ke zariye 0 MB RAM aur 0.001 second mein unit-norm embedding vectors banata hai.
- `write_article`: Writer stage ko execute karta hai. Iske andar `validate_and_heal` function sentence boundary splitter `[.!؟۔]` aur word-level trimming ke zariye target words (e.g. 450 words) ko strictly enforce karta hai.
- `generate(payload, ...)`: Master async function jo Ingestion, Web Search, Dense Retrieval, Audience, SEO, Outline, Writer, aur Review ko sequentially aur parallelly run karta hai.
- Concurrency optimization: `unified_review_reports` aur `write_article` ko `asyncio.gather` se parallel chalata hai jis se 15-20 seconds ki bachat hoti hai.

Representative Code Listing (planner/engine.py):

```
import asyncio
import importlib.util
import os
import re
import time
from pathlib import Path
from uuid import uuid4
from functools import lru_cache
from .contracts import Intake, Article, FAQ, article_text, count_words, validate_article, final_stat
from .ingestion import read_document, fetch_source, discover_sources, chunk_sources
from .review import review_report, unified_review_reports
from .security import public_url

os.environ["OTEL_SDK_DISABLED"] = "true"
os.environ["CREWAI_TELEMETRY_DISABLED"] = "true"
os.environ["TOKENIZERS_PARALLELISM"] = "false"


class FallbackTokenizer:
    def __call__(self, text, add_special_tokens=False, return_offsets_mapping=False, truncation=Fals
        if not text or not text.strip():
            return {"input_ids": [0], "offset_mapping": [(0, 0)]}
        tokens = list(re.finditer(r"\S+", text))
        if not tokens:
            return {"input_ids": [0], "offset_mapping": [(0, 0)]}
        offset_mapping = [m.span() for m in tokens]
        input_ids = [abs(hash(m.group())) % 100000 + 1 for m in tokens]
        return {"input_ids": input_ids, "offset_mapping": offset_mapping}


class FallbackEmbeddingModel:
    """Zero-download fallback embedding model when Hugging Face hub is offline or unreachable."""
    def __init__(self, dim=384):
```

```
        self.dim = dim
        self.max_seq_length = 512
        self.tokenizer = FallbackTokenizer()

    def encode(self, sentences, normalize_embeddings=True, convert_to_numpy=True, batch_size=16, sho
        import hashlib
        import numpy as np
        is_single = isinstance(sentences, str)
        items = [sentences] if is_single else list(sentences)
# ... [Truncated: 290 more lines in full code file]
```

File: planner/transport.py

>> Module Role: LLM Transport, CrewAI Integration & Rate Limiter

Architectural Role & Work of planner/transport.py

transport.py external AI model providers (Groq, Cerebras, Google Gemini) ke sath actual network communication ko handle karta hai. Is file mein rate limiting, token budgeting, self-healing JSON repair, aur CrewAI execution engine mojood hai.

Key Components & Logic:
- `safe_parse_json`: Agar model ka return kiya hua JSON markdown backticks mein ho ya syntax thora sa toot gaya ho (missing brackets), to yeh stack algorithm se brackets balance karke cleanly parse karta hai.
- `retry_policy`: HTTP 429 (Rate Limit) ya 500/503 errors par exponential backoff karta hai aur 'retry-after' header ko respect karta hai.
- `JSONTransport(BaseLLM)`: CrewAI ka custom LLM class jo enforce karta hai `response_format={'type': 'json_object'}`.
- `STAGE_TOKENS`: Har stage ke liye optimized completion token limits (Intake: 400, SEO: 600, Research: 800, Structure: 1200, Writer: 1800) taake Groq 6,000 TPM limit cross na ho aur Cerebras par latency kam ho.

Representative Code Listing (planner/transport.py):

```
import asyncio
import json
import os
os.environ["OTEL_SDK_DISABLED"] = "true"
os.environ["CREWAI_TELEMETRY_DISABLED"] = "true"
os.environ["TOKENIZERS_PARALLELISM"] = "false"
import time
import re
from email.utils import parsedate_to_datetime
from datetime import datetime, timezone
from crewai import Agent, Crew, Task, Process, BaseLLM
from groq import Groq
import httpx
from .security import redact


class ProviderFailure(RuntimeError):
    pass


def safe_parse_json(raw_text: str, schema):
    text = raw_text.strip()
    text = re.sub(r"^```(?:json)?\s*", "", text, flags=re.IGNORECASE)
    text = re.sub(r"\s*```$", "", text).strip()
    try:
        return schema.model_validate_json(text)
    except Exception:
        pass
    start_idx = text.find("{")
    end_idx = text.rfind("}")
    if start_idx != -1 and end_idx != -1 and end_idx > start_idx:
        candidate = text[start_idx:end_idx + 1]
        try:
            return schema.model_validate_json(candidate)
        except Exception:
```

```
            text = candidate
    cleaned = re.sub(r"[\x00-\x08\x0b\x0c\x0e-\x1f]", " ", text)
    cleaned = re.sub(r",\s*([}\]])", r"\1", cleaned)
    cleaned = re.sub(r'"(\w+)\[\]"', r'"\1": []', cleaned)
    try:
        return schema.model_validate(json.loads(cleaned))
    except Exception:
# ... [Truncated: 293 more lines in full code file]
```

File: planner/notebook_stages.py

>> Module Role: Agent Prompts & Domain-Agnostic Stage Runners

Architectural Role & Work of planner/notebook_stages.py

notebook_stages.py har agent stage ke tailored system prompts, rules, aur execution wrappers define karta hai. Yeh file notebook ke stages ka production-isolated version hai.

Key Functions & Logic:
- `select_sources`: Source selection agent ka prompt jo trustworthy domains ko filter karta hai.
- `encode_chunks`: Chunks ke dense vectors banata hai.
- `retrieve_evidence`: Domain-agnostic query vector aur passage vectors ka dot product (cosine similarity) nikalta hai aur Top-5 evidence choose karta hai.
- `run_research`: Factual researcher jo chunk IDs ke sath findings extract karta hai.
- `run_audience`: Reader needs aur foundational assumptions banata hai.
- `run_seo`: Primary, secondary, long-tail keywords aur search intent formulate karta hai.
- `run_structure`: H2/H3 outline banata hai with authentic Urdu script mandate (`اردو رسم الخط`).

Representative Code Listing (planner/notebook_stages.py):

```
import json
import hashlib
from pathlib import Path
from datetime import datetime, timezone
from typing import Literal
from urllib.parse import urlparse
import numpy as np
import requests
from pydantic import BaseModel, ConfigDict, Field

class StrictModel(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)


Language = Literal["English", "Urdu", "Roman Urdu"]
Intent = Literal["informational", "commercial", "transactional", "navigational"]


class EvidenceQuote(StrictModel):
    chunk_id: str
    quote: str = Field(min_length=8, max_length=700)


class StageError(RuntimeError):
    pass


def save_json(path, value):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(".tmp")
    temporary.write_text(
        json.dumps(value, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    temporary.replace(path)
```

```
def normalized(text):
    return " ".join(text.split())


def load_user_documents(request):
# ... [Truncated: 379 more lines in full code file]
```

File: planner/ingestion.py

>> Module Role: Multi-Format Parsing, DuckDuckGo Search & Scraping

Architectural Role & Work of planner/ingestion.py

ingestion.py raw data ko clean, token-bounded factual passages mein convert karta hai. Is file mein multi-format file readers aur web discovery tools shamil hain.

Key Functions & Logic:
- `read_document(file_path)`: PDF (pypdf), Excel (openpyxl), PowerPoint (python-pptx), aur Text/Markdown files ko parse karta hai (max 60,000 chars).
- `fetch_source(url)`: Requests aur BeautifulSoup se live webpage download karta hai aur navbars/footers/scripts nikal kar clean article text banata hai.
- `discover_sources(request)`: DuckDuckGo Search API se live web queries run karke top reference URLs dhoondta hai.
- `chunk_sources(sources, model)`: Text ko 240-word sliding window chunks mein kaat kar unique Chunk IDs (`SRC1-C1`, `SRC1-C2`) assign karta hai.

Representative Code Listing (planner/ingestion.py):

```
import csv
from pathlib import Path
from urllib.parse import urljoin
import requests
from bs4 import BeautifulSoup
from .security import public_url


def read_document(file_path):
    path = Path(file_path)
    sections, notices, used = [], [], 0

    def add(text, location):
        nonlocal used
        remaining = 60000 - used
        if remaining <= 0:
            return
        text = text.strip()[:remaining]
        if text:
            sections.append({"text": text, "location": location})
            used += len(text)

    suffix = path.suffix.lower()
    if suffix in {".txt", ".md", ".csv"}:
        add(path.read_text(encoding="utf-8-sig"), "Document")
    elif suffix == ".pdf":
        from pypdf import PdfReader
        reader = PdfReader(path)
        for number, page in enumerate(reader.pages[:80], 1):
            add(page.extract_text() or "", f"Page {number}")
            if used >= 60000:
                break
        if len(reader.pages) > 80:
            notices.append("Only the first 80 PDF pages were processed.")
    elif suffix == ".xlsx":
        from openpyxl import load_workbook
```

```
            book = load_workbook(path, read_only=True, data_only=True, keep_links=False)
            try:
                for sheet in book:
                    rows = sheet.iter_rows(max_row=1000, max_col=30, values_only=True)
                    add("\n".join(" | ".join(str(v) for v in row if v is not None) for row in rows), she
                    if used >= 60000:
    # ... [Truncated: 112 more lines in full code file]
```

File: planner/review.py

>> Module Role: Quality Audit, Claim Extraction & Hallucination Check

Architectural Role & Work of planner/review.py

review.py pure system ka quality control inspector hai. Iska kaam writer aur planning stages ke claims ko evidence ke samnay rakh kar check karna hai.

Key Functions & Logic:
- `units_for(target, report)`: Draft ke har jumlay ya section ko verifiable atomic units mein divide karta hai.
- `evidence_catalog(evidence)`: Tamam retrieved chunks ka lookup index banata hai.
- `review_report`: LLM Auditor ko har claim aur evidence chunks deta hai. Model classify karta hai ke claim 'supported' hai ya 'unsupported'.
- `unified_review_reports`: Research, SEO, aur Outline ko ek single combined pass mein audit karta hai taake extra API calls aur time bach sake.

Representative Code Listing (planner/review.py):

```
from .contracts import final_status


def evidence_catalog(evidence):
    catalog = {}
    for item in evidence:
        text = " ".join(item["text"].split())
        for i, start in enumerate(range(0, len(text), 450), 1):
            quote = text[start:start + 650]
            if len(quote) >= 8:
                catalog[f"{item['chunk_id']}:E{i}"] = {"chunk_id": item["chunk_id"], "quote": quote}
    return catalog


def units_for(target, report):
    if target == "research":
        units = {f"finding.{i}": f["statement"] for i, f in enumerate(report["findings"])}
        units["missing_information"] = "\n".join(report["missing_information"]) or "No missing infor
        return units
    if target == "seo":
        return {k: str(v) for k, v in report.items() if k != "output_language"}
    units = {k: report[k] for k in ("title", "meta_description")}
    if target == "article":
        units["introduction"] = report["introduction"]
        for i, section in enumerate(report["sections"]):
            units[f"section.{i}"] = section["heading"] + "\n" + section["body"]
        for i, faq in enumerate(report["faqs"]):
            units[f"faq.{i}"] = faq["question"] + "\n" + faq["answer"]
    else:
        units["h1"] = report["h1"]
        units["keywords"] = ", ".join(report["suggested_keywords"])
        for i, section in enumerate(report["outline"]):
            units[f"heading.{i}"] = section["h2"] + " / " + " / ".join(section["h3"])
            for j, note in enumerate(section["coverage_notes"]):
                units[f"coverage.{i}.{j}"] = note
        units.update({f"faq.{i}": q for i, q in enumerate(report["faqs"])})
        units["limitations"] = "\n".join(report["limitations"])
    return units
```

```
RULES = """Review every required location. Return assessments keyed by the exact location.
Use English explanations. Assess meaning, not verbatim wording.
# ... [Truncated: 313 more lines in full code file]
```

File: planner/exports.py

>> Module Role: Publishing Engine: Markdown & Typeset Vector PDF

Architectural Role & Work of planner/exports.py

exports.py final generated output ko user ke download karne ke liye format karta hai. Is file mein Markdown generation aur FPDF2 vector PDF creation with Urdu HarfBuzz font shaping implement ki gayi hai.

Key Functions & Logic:
- `content_markdown(result)`: Title, Intro, Sections, aur FAQs ko clean Github Flavored Markdown mein convert karta hai.
- `download_markdown(result)`: Content ke sath SEO metadata, Review limitations, aur Reference URLs append karta hai.
- `make_pdf(result)`: FPDF2 document create karta hai, Noto Sans aur Noto Naskh Arabic fonts load karta hai, `pdf.set_text_shaping(True)` se authentic Urdu nastaliq/naskh ligatures jorta hai, aur running header/footer ke sath styled PDF byte stream return karta hai.

Representative Code Listing (planner/exports.py):

```
import re
from pathlib import Path

STATUS_LABELS = {
    "automated_review_passed": "Automated review passed",
    "needs_revision": "Draft needs attention",
    "draft_without_sources": "Draft without retrieved sources",
    "review_incomplete": "Review incomplete",
}


def safe_markdown(text):
    text = re.sub(r"!\[[^\]]*\]\([^)]*\)", "", text)
    text = re.sub(r"<[^>]+>", "", text)
    return re.sub(r"\[([^\]]+)\]\((?!https?://)[^)]*\)", r"\1", text)


def refs(ids):
    return "\n\nSources: " + ", ".join(dict.fromkeys(ids)) if ids else ""


def content_markdown(result):
    article, brief = result.get("article"), result["brief"]
    if article:
        parts = ["# " + article["title"], article["introduction"]]
        for section in article["sections"]:
            parts.extend(["## " + section["heading"], section["body"] + refs(section["chunk_ids"])])
        if article.get("faqs"):
            parts.append("## Frequently asked questions" if article["output_language"] != "Urdu" els
            for faq in article["faqs"]:
                parts.extend(["### " + faq["question"], faq["answer"] + refs(faq["chunk_ids"])])
    else:
        parts = ["# " + brief["title"], brief["meta_description"], "## Main heading", brief["h1"]]
        for section in brief["outline"]:
            parts.append("## " + section["h2"])
            parts.extend("### " + heading for heading in section["h3"])
            parts.extend("- " + note for note in section["coverage_notes"])
```

```
            if section["chunk_ids"]:
                parts.append(refs(section["chunk_ids"]).strip())
        if brief.get("faqs"):
            parts.append("## Frequently asked questions")
            parts.extend("- " + q for q in brief["faqs"])
# ... [Truncated: 95 more lines in full code file]
```

File: planner/security.py

>> Module Role: Cybersecurity Guardrails & SSRF Prevention

Architectural Role & Work of planner/security.py

security.py system ki application-level security enforce karta hai. Iska kaam prompt injection, Server-Side Request Forgery (SSRF), aur API key leakage ko rokna hai.

Key Functions & Logic:
- `clean_url(url)` & `public_url(url)`: Validate karta hai ke URLs valid web links hon aur private LAN IPs (127.0.0.1, 10.x.x.x, 192.168.x.x) ko block karta hai taake koi attacker local server access na kar sake.
- `validate_upload(name, data)`: File extensions (.pdf, .xlsx, .pptx, .txt) aur unke magic bytes verify karta hai taake executable malware upload na ho sake.
- `redact(text, key)`: Logs aur error messages mein se Groq, Gemini, ya Cerebras ki secret API keys ko mask (`***`) karta hai taake credentials leak na hon.

Representative Code Listing (planner/security.py):

```
import ipaddress
import re
import socket
from urllib.parse import urlsplit, urlunsplit
from pathlib import Path
from zipfile import ZipFile, BadZipFile
from io import BytesIO

MAX_FILE_BYTES = 10 * 1024 * 1024
ALLOWED_EXTENSIONS = {".pdf", ".txt", ".md", ".csv", ".xlsx", ".pptx"}


def public_url(url):
    parsed = urlsplit(url.strip())
    if parsed.scheme not in {"http", "https"} or not parsed.hostname:
        raise ValueError("Enter a public HTTP or HTTPS URL.")
    if parsed.username or parsed.password or parsed.port not in (None, 80, 443):
        raise ValueError("URLs with credentials or custom ports are not supported.")
    try:
        addresses = socket.getaddrinfo(parsed.hostname, parsed.port or 443, type=socket.SOCK_STREAM)
        if not addresses or any(not ipaddress.ip_address(a[4][0]).is_global for a in addresses):
            raise ValueError("Only public websites are supported.")
    except socket.gaierror as exc:
        raise ValueError("The website address could not be resolved.") from exc
    return urlunsplit(parsed._replace(fragment=""))


def safe_name(name):
    return re.sub(r"[^\w. -]", "_", Path(name.replace("\\", "/")).name)[:100] or "document"


def validate_upload(name, data):
    suffix = Path(name).suffix.lower()
    if suffix not in ALLOWED_EXTENSIONS:
        raise ValueError("Supported files: PDF, TXT, MD, CSV, XLSX, PPTX.")
    if not data or len(data) > MAX_FILE_BYTES:
        raise ValueError("Each file must contain data and be at most 10 MB.")
    if suffix == ".pdf" and not data.lstrip().startswith(b"%PDF-"):
```

```
            raise ValueError("This file is not a valid PDF.")
        if suffix in {".xlsx", ".pptx"}:
            try:
                with ZipFile(BytesIO(data)) as archive:
    # ... [Truncated: 13 more lines in full code file]
```

File: planner/sample.py

>> Module Role: Zero-Cost Demo Data Loader

Architectural Role & Work of planner/sample.py

sample.py demonstration aur fast testing ke liye banaya gaya hai. Jab kisi user ke paas API key na ho ya developer ko UI test karni ho, to yeh pre-calculated sample package return karta hai.

Key Logic:
- `sample_result()`: Aik realistic Machine Learning SEO plan return karta hai jismein Outline, Audience, SEO Keywords, Review Status, aur References pehle se structured hotay hain. Is par click karne se 0 API calls lagti hain aur 0.1 second mein dashboard populate ho jata hai.

Representative Code Listing (planner/sample.py):

```
import json
from pathlib import Path


def sample_result():
    sample_file = Path(__file__).resolve().parents[1] / "sample_data" / "sample_run.json"
    if sample_file.exists():
        try:
            data = json.loads(sample_file.read_text(encoding="utf-8"))
            # Transform into the contract format expected by show_results
            sources_dict = {}
            for s in data.get("sources", []):
                sources_dict[s["source_id"]] = {
                    "title": s.get("title", "Reference"),
                    "url": s.get("url")
                }
            evidence_list = []
            for e in data.get("evidence", []):
                chunk_id = e.get("chunk_id", "")
                src_id = chunk_id.split("-")[0] if "-" in chunk_id else "S1"
                evidence_list.append({
                    "source_id": src_id,
                    "chunk_id": chunk_id,
                    "location": e.get("location", "Webpage"),
                    "text": e.get("text", "")
                })

            review_dict = {}
            for target, r in data.get("review", {}).items():
                assessments = []
                for a in r.get("assessments", []):
                    assessments.append({
                        "location": a.get("location", ""),
                        "verdict": a.get("verdict", "editorial"),
                        "explanation": a.get("explanation", ""),
                        "citations": a.get("citations", [])
                    })
                review_dict[target] = {
                    "issues": r.get("issues", []),
                    "assessments": assessments,
                    "limitations": r.get("limitations", [])
                }
```

```
# ... [Truncated: 85 more lines in full code file]
```